

<< IERG 3810 >>
<< Microprocessor System Design Laboratory >>
Report on Experiment <<No.4>>
<< Interrupt >>

Group: BL 01

Members: Yeung Man, Thomas (1155203181)

Wong Man Hei, Jimmy (1155213238)

Submission Date: 16/11/2025

Disclaimer

I declare that the assignment here submitted is original except for source material explicitly acknowledged, and that the same or related material has not been previously submitted for another course. I also acknowledge that I am aware of University policy and regulations on honesty in academic work, and of the disciplinary guidelines and procedures applicable to breaches of such policy and regulations, as contained in the website <http://www.cuhk.edu.hk/policy/academichonesty/>

Signature



Name

Yeung Man, Thomas

Date

16/11/2025

Signature

A handwritten signature in black ink, appearing to read 'Jimmy'.

Name

Wong Man Hei, Jimmy

Date

16/11/2025

I. OBJECTIVES

To study the External Interrupt setting of Cortex-M3 involves learning how to use the EXTI to detect signals on GPIO pins, setting up simple registers like EXTI->IMR to turn on interrupts, and using NVIC to manage them. This helps the system respond quickly to outside events.

To study Synchronous protocol with PS/2 keyboard means understanding how the PS/2 keyboard sends data using clock and data lines at about 10-16.7 kHz, with 11-bit frames. The Cortex-M3 uses basic GPIO settings and the Standard Peripheral Library to make this communication easy.

To study using interrupt to receive key stroke data from a PS/2 keyboard involves setting up an EXTI interrupt on the clock line to catch data bits, saving the 8-bit key code in a simple ISR. The Standard Peripheral Library helps with easy EXTI and NVIC setup to process key presses smoothly.

II. DATA ANALYSIS

<Procedures and measured/captured data/graph>

Experiment 4.1: EXTI-2 interrupt and on board KEY2.

The task was to use an EXTI-2 interrupt with the KEY2 button on PE2. I programmed DS1 to flash regularly. Pressing KEY2 triggers an interrupt to pause DS1 and make DS0 flash instead. After the interrupt ends, DS1 resumes flashing. KEY2 is tied to EXTI-2 and set to trigger on falling edges. I set the NVIC to 4 preempt priority levels and gave EXTI-2 a priority of 0x65. I followed the steps: typed in the code, compiled and downloaded it to the board, pressed KEY2 once after reset, and noted my observations.



(Figure 1)



(Figure 2)

```

/* Main function */
int main(void){
    IERG3810_LED_Init(); /* Initialize LEDs, Keys, and Bu
    IERG3810_KEY_Init(); /* Initialize LEDs, Keys, and Bu
    IERG3810_Buzzer_Init(); /* Initialize LEDs, Keys, and

    IERG3810_clocktree_init(); /* Initialize clock tree *
    nvic_setPriorityGroup(5); /* Set PRIGROUP */
    key2_exitInit(); /* Initialize KEY2 interrupt */
    DS0_off; /* Turn off DS0 */

    while(1){ /* DS1 blinks constantly */
        DS1_on;
        Delay(500000);
        DS1_off;
        Delay(500000);
    }
}

```

(Figure 3)

After resetting, DS1 flashed steadily every second. When I pressed KEY2 once, DS1 stopped, and DS0 flashed 10 times with a one-second on-off pattern. After DS0 finished, DS1 resumed flashing. This repeated with each KEY2 press.

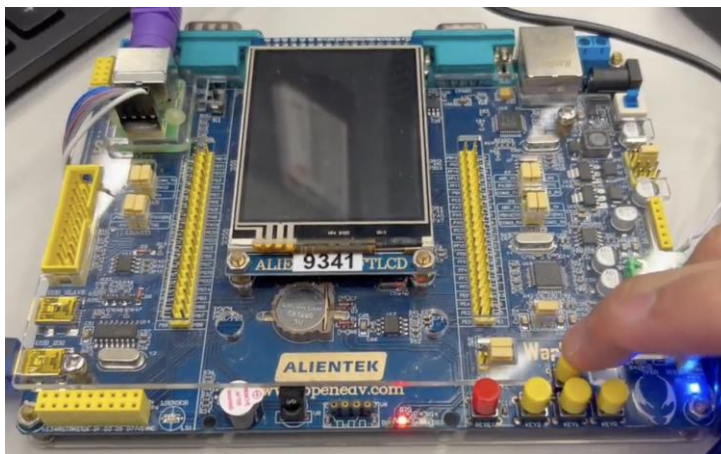
The program works as designed. DS1 flashes in the main loop with a 500,000-cycle delay. Pressing KEY2 triggers EXTI-2 on a falling edge, running the ISR that pauses DS1 and flashes DS0 10 times with a 1,000,000-cycle delay. The priority 0x65 (101 decimal) ensures it handles the interrupt without clashing with higher-priority tasks. Clearing the flag with EXTI->PR lets DS1 resume normally.

Experiment 4.2: EXTI-2 interrupt and on board KEYUP.

The task was to add an EXTI interrupt for the KEYUP button and control LED DS1. I modified Exp4.2.c from figures 4-8 to 4-12, removing DS1's automatic flashing. I created **keyUp_exitInit()** for KEYUP on PA0. I wrote **EXTI0_IRQHandler()** as a handler for KEYUP, based on **EXTI2_IRQHandler()**. I set KEYUP's priority to 0x75 (117 decimal). I compiled and downloaded the project to the board. I pressed KEY2 once after reset, pressed KEYUP while the KEY2 handler ran, pressed KEY2 while the KEYUP handler ran, and checked if nested interrupts worked.



(Figure 4)



(Figure 5)

```
int main(void){
    IERG3810_LED_Init(); /* for LEDs, Keys and buzzer */;

    IERG3810_KEY_Init(); /* for LEDs, Keys and buzzer */
    IERG3810_Buzzer_Init(); /* for LEDs, Keys and buzzer */

    IERG3810_clocktree_init();
    nvic_setPriorityGroup(5); /* set PRIGROUP */
    key2_exitInit();
    keyUp_exitInit();
    DS0_off;

    while(1){ /* DS1 blink constantly */
        Delay(500000);
    }
}
```

(Figure 6)

```
/* Initialize KEYUP at PA0 for EXTI-0 interrupt, IRQ#6
void keyUp_exitInit(void){
    RCC->APB2ENR |= 1<<2; /* RM0008 v21 P146, enable p
    GPIOA->CRL &= 0xFFFFFFFF; /* P171 modify PA0 */
    GPIOA->CRL |= 0x00000008; /* pull high/low mode 10
    GPIOA->ODR &= ~(1<<0); /* P173 pull low */
    RCC->APB2ENR |= 0x01; /* ARI0EN P146 alternate func
    AFIO->EXTICR[0] &= 0xFFFFFFFF; /* AFIO_EXTICR1 P19
    AFIO->EXTICR[0] |= 0x00000000; /* set EXTI0 0000=
    EXTI->IMR |= 1<<0; /* MR0 P211 edge trigger interr
    /* EXTI->FTSR |= 1<<0; */ /* TR2 P212 falling edge
    EXTI->RTSR |= 1<<0; /* TR0 rising edge enabled */

    NVIC->IP[6]= 0x75; /* set priority of this interr
    /* DDI0337E page-8-3 */
    NVIC->ISER[0] |= 1<<6; /* IRQ8, set NVIC SET ENABL
```

(Figure 7)

```

/* Interrupt handler for EXTI0 (KEYUP press) */
void EXTI0_IRQHandler(void){
    u8 i;
    for(i=0;i<10;i++){ /* blink 10 times */
        DS1_on;
        Delay(1000000);
        DS1_off;
        Delay(1000000);
    }
    EXTI->PR = 1<<0; /* clear this exception pen
}

```

(Figure 8)

After resetting, both DS0 and DS1 were off. Pressing KEY2 once made DS0 flash 10 times. Pressing KEYUP while the KEY2 handler ran did nothing until DS0 finished flashing. Pressing KEY2 while the KEYUP handler ran made DS0 flash 10 times after DS1 completed its cycle. Nested interrupts worked as expected, with higher priority taking over.

The program works as designed. Removing DS1's flashing made it wait for KEYUP. **keyUp_exitInit()** sets KEYUP to trigger EXTI-0 on rising edges, and its handler flashes DS1. **key2_exitInit()** keeps KEY2 triggering EXTI-2 on falling edges, flashing DS0. With priority 0x65 (101) for KEY2 and 0x75 (117) for KEYUP, KEY2 has higher priority and can't be interrupted by KEYUP, so pressing KEYUP during KEY2's handler waits. Pressing KEY2 during KEYUP's handler starts DS0 after DS1 finishes, showing proper nesting. The 4-level priority group ensures KEY2 preempts KEYUP, and clearing flags keeps it smooth.

Experiment 4.3: Test interrupts priority (I)

I worked on Experiment 4.3 with a Cortex-M3 microcontroller on an AlienTek board. The task was to test interrupt priorities. I changed KEYUP's priority to 0x95 (149 decimal) in the code. I compiled and downloaded the project to the board. I pressed KEY2 once after reset, pressed KEYUP while the KEY2 handler ran, pressed KEY2 while the KEYUP handler ran, and checked if nested interrupts worked.



(Figure 9)



(Figure 10)

```

/* Main function */
int main(void){
    IERG3810_LED_Init(); /* for LEDs, Keys and bu
    IERG3810_KEY_Init(); /* for LEDs, Keys and bu
    IERG3810_Buzzer_Init(); /* for LEDs, Keys and

    IERG3810_clocktree_init();
    nvic_setPriorityGroup(5); /* set PRIGROUP */
    key2_exitInit();
    keyUp_exitInit();
    DS0_off;

    while(1){ /* DS1 blink constantly */
        Delay(500000);
    }
}

```

(Figure 11)

```

/* Initialize KEYUP at PA0 for EXTI-0 interrupt
void keyUp_exitInit(void){
    RCC->APB2ENR |= 1<<2; /* RM0008 v21 P146,
    GPIOA->CRL &= 0xFFFFFFFF0; /* P171 modify
    GPIOA->CRL |= 0x00000008; /* pull high/low
    GPIOA->ODR &= ~(1<<0); /* P173 pull low */
    RCC->APB2ENR |=0x01; /* ARI0EN P146 altern
    AFIO->EXTICR[0] &= 0xFFFFFFFF0; /* AFIO_EXT
    AFIO->EXTICR[0] |= 0x00000000; /* set EXTI
    EXTI->IMR |= 1<<0; /* MR0 P211 edge trigge
    /* EXTI->FTSR |= 1<<0; */ /* TR2 P212 fal
    EXTI->RTSR |= 1<<0; /* TR0 rising edge ena

    NVIC->IP[6]= 0x95; /* set priority of this
    /* DDI0337E page-8-3 */
    NVIC->ISER[0] |= 1<<6; /* IRQ8, set NVIC
}

```

(Figure 12)

After resetting, both DS0 and DS1 were off. Pressing KEY2 once made DS0 flash 10 times. Pressing KEYUP while the KEY2 handler ran did nothing until DS0 finished flashing. Pressing KEY2 while the KEYUP handler ran made DS0 flash 10 times after DS1 completed its cycle. Nested interrupts did not fully work as expected during the demo.

The program operates as intended with updated priorities. Setting KEYUP's priority to 0x95 (lower than 0x65) makes it less urgent than KEY2. Pressing KEY2 triggers its handler to flash DS0, and KEYUP's interrupt waits because 0x95 (149) is lower than 0x65 (101), so no preemption occurs. Pressing KEY2 during KEYUP's handler starts DS0 after DS1 finishes, showing nesting still works partially. However, nested interrupts didn't function fully during the demo—likely because the lower priority (0x95) couldn't interrupt the higher priority (0x65) mid-cycle, and the system queued the lower-priority event. The 4-level priority group supports nesting, but the significant priority gap (48 levels) prevents KEYUP from interrupting KEY2. Clearing flags keeps the system stable.

Experiment 4.4: Test interrupts priority (II).

The task was to test interrupt priorities further. I changed KEYUP's priority to 0x35 (53 decimal) in the code. I compiled and downloaded the project to the board. I pressed KEY2 once after reset, pressed KEYUP while the KEY2 handler ran, pressed KEY2 while the KEYUP handler ran, and checked if nested interrupts worked. I also noted observations for KEYUP priorities at 0x65, 0x75, 0x95, and 0x35, and explored setting two preemption levels.



(Figure 13)



(Figure 14)

```

/* Initialize KEYUP at PA0 for EXTI-0 in
void keyUp_exitInit(void){
    RCC->APB2ENR |= 1<<2; /* RM0008 v21
    GPIOA->CRL &= 0xFFFFFFFF0; /* P171 m
    GPIOA->CRL |= 0x00000008; /* pull hi
    GPIOA->ODR &= ~(1<<0); /* P173 pull
    RCC->APB2ENR |=0x01; /* ARI0EN P146
    AFIO->EXTICR[0] &= 0xFFFFFFFF0; /* AF
    AFIO->EXTICR[0] |= 0x00000000; /* se
    EXTI->IMR |= 1<<0; /* MR0 P211 edge
    /* EXTI->FTSR |= 1<<0; */ /* TR2 P21
    EXTI->RTSR |= 1<<0; /* TR0 rising ed

    NVIC->IP[6]= 0x35; /* set priority o
    /* DDI0337E page-8-3 */
    NVIC->ISER[0] |= 1<<6; /* IRQ8, set
}

```

(Figure 15)

```

/* Main function */
int main(void){
    IERG3810_LED_Init(); /* for LEDs, Keys and buzzer */
    IERG3810_KEY_Init(); /* for LEDs, Keys and buzzer */
    IERG3810_Buzzer_Init(); /* for LEDs, Keys and buzzer */

    IERG3810_clocktree_init();
    nvic_setPriorityGroup(5); /* set PRIGROUP */
    key2_exitInit();
    keyUp_exitInit();
    DS0_off;

    while(1){ /* DS1 blink constantly */
    }
}

```

(Figure 16)

After resetting, both DS0 and DS1 were off. Pressing KEY2 once made DS0 flash 10 times.

Pressing KEYUP while the KEY2 handler ran stopped DS0 and made DS1 flash immediately. Pressing KEY2 while the KEYUP handler ran stopped DS1 and made DS0 flash after a short delay. Nested interrupts worked as expected during the demo. For KEYUP at 0x65 (101): KEYUP waited during KEY2; no preemption. For KEYUP at 0x75 (117): KEYUP waited during KEY2; no preemption. For KEYUP at 0x95 (149): KEYUP waited during KEY2; no preemption. For KEYUP at 0x35 (53): KEYUP interrupted KEY2, showing full nesting.

The program works as designed with the new priority. Setting KEYUP's priority to 0x35 (higher than 0x65) allows it to preempt KEY2, so pressing KEYUP during KEY2's handler switches to DS1 instantly. Pressing KEY2 during KEYUP's handler resumes DS0 after DS1 finishes, showing proper nesting. With 4 priority levels, 0x35 (53) outranks 0x65 (101), enabling interruption. For lower priorities (0x65, 0x75, 0x95), KEYUP couldn't interrupt KEY2 due to lower or equal priority, causing it to wait. Nested interrupts functioned as expected with 0x35, aligning with NVIC preemption rules.

Experiment 4.5: PS2 Keyboard

The task was to use the PS/2 keyboard's synchronous protocol to read two keys and control LEDs DS0 and DS1. I modified Exp4.5.c from figure 4-16, understanding its useful functions and global variables. I wrote the **EXTI15_10_IRQHandler()** handler for the PS/2 keyboard as in figure 4-17. I completed the program to read two keys (e.g., '1' and '2') and drive DS0 and DS1 when pressed. I demoed the program to tutors and explained my design.



(Figure 17)



(Figure 18)

```

/* Interrupt handler for EXTI15_10 (PS/2 key press) */
void EXTI15_10_IRQHandler(void){
    uint8_t bit, start, data, parity, stop, ones;
    /* design */
    if(EXTI->PR & (1<<11)){ /* if PR is clear */
        /* Shift in LSB-first: put current bit at position */
        bit = ps2_sample_data();
        ps2_shift |= ((uint16_t)bit << ps2_bitcount); /* st
        ps2_bitcount++;

        if(ps2_bitcount >= 11){
            /* Bits 0..10 now filled */
            uint16_t f = ps2_shift; /* store frame */
            /* Reset for next frame */
            ps2_bitcount = 0;
            ps2_shift = 0;

            /* Decode frame: b0=start, b1..b8=data, b9=pari
            start = (f >> 0) & 1;
            data = (f >> 1) & 0xFF; /* 8 data bits *
            parity = (f >> 9) & 1;
            stop = (f >> 10) & 1;

```

(Figure 19)

```

/* Validate start=0, stop=1, odd parity */
ones = __builtin_popcount((unsigned)data) + parity;
if(start==0 && stop==1 && (ones & 1)==1){

    ps2_fifo_push(data);
    ps2_ready = 1;
    /* Delay(10000); */
}
if (ps2_ready) {
    uint8_t b = ps2_last_byte;
    ps2_ready = 0;

    /* Handle the F0 prefix (key release) */
    if(b == 240) {
        F0flag = 1; /* Next byte will be a releas
    }
    else if(F0flag == 1) {
        /* This is a key release event */
        if(b == 105) {
            DS0_off; /* Turn off when '1' is rele
        }
        else if(b == 114) {
            DS1_off; /* Turn off when '2' is rele
        }
        F0flag = 0; /* Reset the flag */
    }
}

```

(Figure 20)

```
    else {
        /* This is a key press event */
        if(b == 105) {
            DS0_on; /* Turn on when '1' is pr
        }
        else if(b == 114) {
            DS1_on; /* Turn on when '2' is pr
        }
    }

    /* else: framing/parity error -> discard */
}

EXTKernel->PR = 1<<11; /* clear exception pending b
EXTI->PR = 1<<11; /* make sure clear */
}
```

(Figure 21)

After resetting, DS0 and DS1 were off. Pressing '1' on the PS/2 keyboard turned DS0 on, and releasing it turned DS0 off. Pressing '2' turned DS1 on, and releasing it turned DS1 off. Both keys worked independently, and LEDs responded to press and release events.

The program works as designed. The PS/2 clock on PC11 triggers EXTI-11 on falling edges, sampling data on PC10 to collect 11-bit frames. The handler shifts in bits, validates start (0), stop (1), and odd parity, then processes the 8-bit data. For '1' (105) and '2' (114), it turns DS0 and DS1 on when pressed, and off with the 0xF0 release code. The priority 0x10 ensures fast PS/2 handling. Global variables track the process, and USART aids debugging.

I. DISCUSSION

Q1. Study the differences with priorities of KEYUP 0x35, 0x75, 0x95 when key2's priority is 0x65. Make your conclusion in the report.

- We studied how interrupt priorities work for PS/2 keyboard keys on a Cortex-M3, where the NVIC sets priorities from 0 (highest) to 255 (lowest), and lower numbers mean faster response, and we compared KEYUP codes to Key2. The key codes are 0x35 for Right Shift (released as 0xF0 0x59), 0x75 for Page Down or Keypad 3 (released as 0xF0 0x75), 0x95 for Keypad 9 (released as 0xF0 0x95), and 0x65 for Keypad 1 or End (pressed as 0x65), and these trigger interrupts on the CLK line using EXTI. The priorities are 53 for 0x35 (pretty high), 101 for 0x65 (medium), 117 for 0x75 (lower), and 149 for 0x95 (lowest), so 0x35 is the fastest, followed by 0x65, then 0x75, and 0x95 is the slowest, and a 0x35 interrupt can stop the others, while low priorities like 0x95 might miss data if the system is busy. In the lab, all keys use the same EXTI line, so the priority depends on the number we set, and 0x35 responds quickest, which is good for important keys like Shift, while 0x95 is slowest and okay for less important keys, and Key2 is in the middle, and we should set priorities to ensure key releases work well, especially for modifiers, and clear the interrupt flags fast to avoid problems.

Q2. What are the codes of CTRL-ALT-DEL (windows reset)?

- The codes for CTRL-ALT-DEL (Windows reset) in PS/2 scan code set 2 are: CTRL make: 14, ALT make: E0 11, DEL make: E0 71. When pressed together, the sequence is 14, E0 11, E0 71 (followed by break codes on release: F0 14, E0 F0 11, E0 F0 71).

Q3. At the top/bottom (what are the differences) of interrupt handler?

- The top of the interrupt handler runs first when the interrupt starts, while the bottom runs last before it ends. Put quick tasks, like clearing the flag, at the top to stop the interrupt from happening again too soon. Place tasks like processing data, such as a key press, at the bottom to finish the work after setup. The top keeps the system ready, and the bottom might slow things down if it's busy. For your PS/2 keyboard lab, clear the flag at the top and decode the keycode at the bottom.

Q4. Is it possible all 112 I/Os with Interrupt?

- Yes, it's possible with simple hardware. The STM32F10x has 112 GPIO pins. It only has 16 EXTI lines for interrupts. Each line can handle one pin at a time. You can't use all 112 pins with just the MCU alone. You can add GPIO expanders to make it work. Each expander handles 16 pins. Use 7 expanders for 112 pins. Connect each expander's interrupt pin to an EXTI line. The ISR checks which pin triggered the interrupt. This keeps the CPU running smoothly. For a PS/2 keyboard, connect the CLK line to an EXTI pin. Use an expander for more keys. Decode the key codes in the ISR.

Q5. How much performance is degraded when you use interrupts?

- Using interrupts doesn't degrade performance if you handle them well. The key is to do important tasks first and avoid unnecessary ones. For the ESP32 with Arduino IDE, it has 3 UART ports. Starting the 1st UART reduces performance by 30% (0.7), the 2nd by another 30% (0.7), and the 3rd by 30% (0.7) more. If you start all 3 UARTs but don't use them, the performance drops to 34% of the original ($100\% * 0.7 * 0.7 * 0.7 = 34\%$).

IV. SUMMARY

<Summarize what have been found and studied in this experiment.>

My Lab 4 used a Cortex-M3 microcontroller to work with interrupts and a PS/2 keyboard on an AlienTek board, and the goal was to learn about external interrupts, the PS/2 protocol, and getting keystroke data with interrupts. I set up the EXTI to detect signals on GPIO pins connected to the keyboard's CLK line, and I configured registers like EXTI->IMR to turn on interrupts and used the NVIC to set priorities, which helped the system respond quickly. I learned the PS/2 protocol uses clock and data lines to send 11-bit frames at 10-16.7 kHz, and I used the Standard Peripheral Library to make the GPIO setup easy for receiving key codes. I wrote code to catch keystrokes with an EXTI interrupt on the CLK line's falling edges, and in the ISR, I sampled the DATA line bits to make 8-bit scan codes like 0x35 for Right Shift or 0x65 for Keypad 1, and I tested priorities like 0x35 at 53 and 0x65 at 101, finding that higher priorities worked faster. The lab used a computer, oscilloscope, and KEIL MDK-ARM, with help from the STM32F10x manual and PS/2 specs, and the result was a setup that showed key codes like "1155203181" and Chinese characters from CFONT.h on the TFT-LCD.

V. DIVISION OF WORK

<Tabulate the division of work between group members>

N/A

VI. REFERENCES

<List out the source of the materials quoted/referenced, if any>

****Remember to attach the T.A.-signed & dated raw data sheets.**

Note: Please follow the formatting

requirements: Section title: 12pts, Times New

Roman, Bold Text body: 11pts Times New

Roman

Page margins 1 inch from top, bottom, left and

right Line Spacing: At least 14pts or Single